

MGT 205 in class - Property

Keman Xiang

Overview

In this project, we will use a data set of property values from 2007 - 2019. The data contain sales prices for houses and units with 1, 2, 3, 4, and 5 bedrooms.

The data are: date of sale; price in dollars; property type (unit or house); number of bedrooms

START HERE

Run the code block below.

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.4
## ✓ forcats    1.0.0      ✓ stringr    1.5.1
## ✓ ggplot2    3.4.4      ✓ tibble     3.2.1
## ✓ lubridate  1.9.3      ✓ tidyr      1.3.0
## ✓ purrr      1.0.2
## — Conflicts — tidyverse_conflicts() —
## * dplyr::filter() masks stats::filter()
## * dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Load the property.csv file into a data frame.

```
prop <- read_csv('property.csv')
```

```
## Rows: 347 Columns: 6
## — Column specification —————
## Delimiter: ","
## chr (2): saledate, type
## dbl (2): price, bedrooms
## lgl (2): V1, V2
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

1. How many rows and columns are there in the data set?

```
dim(prop)
```

```
## [1] 347 6
```

2. Calculate the summary statistics for the “price” column.

```
head(prop)
```

```
## # A tibble: 6 × 6
##   saledate    price type bedrooms V1    V2
##   <chr>      <dbl> <chr>    <dbl> <lgl> <lgl>
## 1 30/06/2007 421291 house      3 NA    NA
## 2 30/06/2007 548969 house      4 NA    NA
## 3 30/06/2007 368817 unit       2 NA    NA
## 4 30/06/2008 441854 house      2 NA    NA
## 5 30/06/2008 419628 house      3 NA    NA
## 6 30/06/2008 559580 house      4 NA    NA
```

```
summary(prop$price)
```

```
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   NA's
## 316751 427681  507596  547434 626106 1017752     1
```

3. Remove the unwanted variables V1 and V2 from the data set. Show the head() of your data set to

confirm that the unwanted variables are gone.

```
prop <- prop [,-(5:6)]  
head(prop)
```

```
## # A tibble: 6 × 4  
##   saledate    price type  bedrooms  
##   <chr>      <dbl> <chr>    <dbl>  
## 1 30/06/2007 421291 house      3  
## 2 30/06/2007 548969 house      4  
## 3 30/06/2007 368817 unit       2  
## 4 30/06/2008 441854 house      2  
## 5 30/06/2008 419628 house      3  
## 6 30/06/2008 559580 house      4
```

4a. Are there any NA values in the data set?

```
any(is.na(prop))
```

```
## [1] TRUE
```

4b. If any NA values are in the “price” column, replace it with the mean price.

```
prop$price[which(is.na(prop$price))] <- mean(prop$price, na.rm=TRUE)
```

4c. If any NA values are in any of the other columns, remove the entire row from the data set.

```
prop<- na.omit(prop)
```

4d. Show that there are no more NA values in the data set.

```
any(is.na(prop))
```

```
## [1] FALSE
```

5. Rename the “saledate” column to “date”. Show the head() of your data set to confirm the change.

```
colnames(prop)[1] <- 'date'  
head(prop)
```

```
## # A tibble: 6 × 4  
##   date      price type  bedrooms  
##   <chr>    <dbl> <chr>    <dbl>  
## 1 30/06/2007 421291 house      3  
## 2 30/06/2007 548969 house      4  
## 3 30/06/2007 368817 unit        2  
## 4 30/06/2008 441854 house      2  
## 5 30/06/2008 419628 house      3  
## 6 30/06/2008 559580 house      4
```

6. Run the code below to convert the variable “bedrooms” into a factor variable using the as.factor() function. Replace “prop” in the code with whatever you called your data set. You do not need to do anything else in this question after you edit and run the code.

```
prop$bedrooms<-as.factor(prop$bedrooms)
```

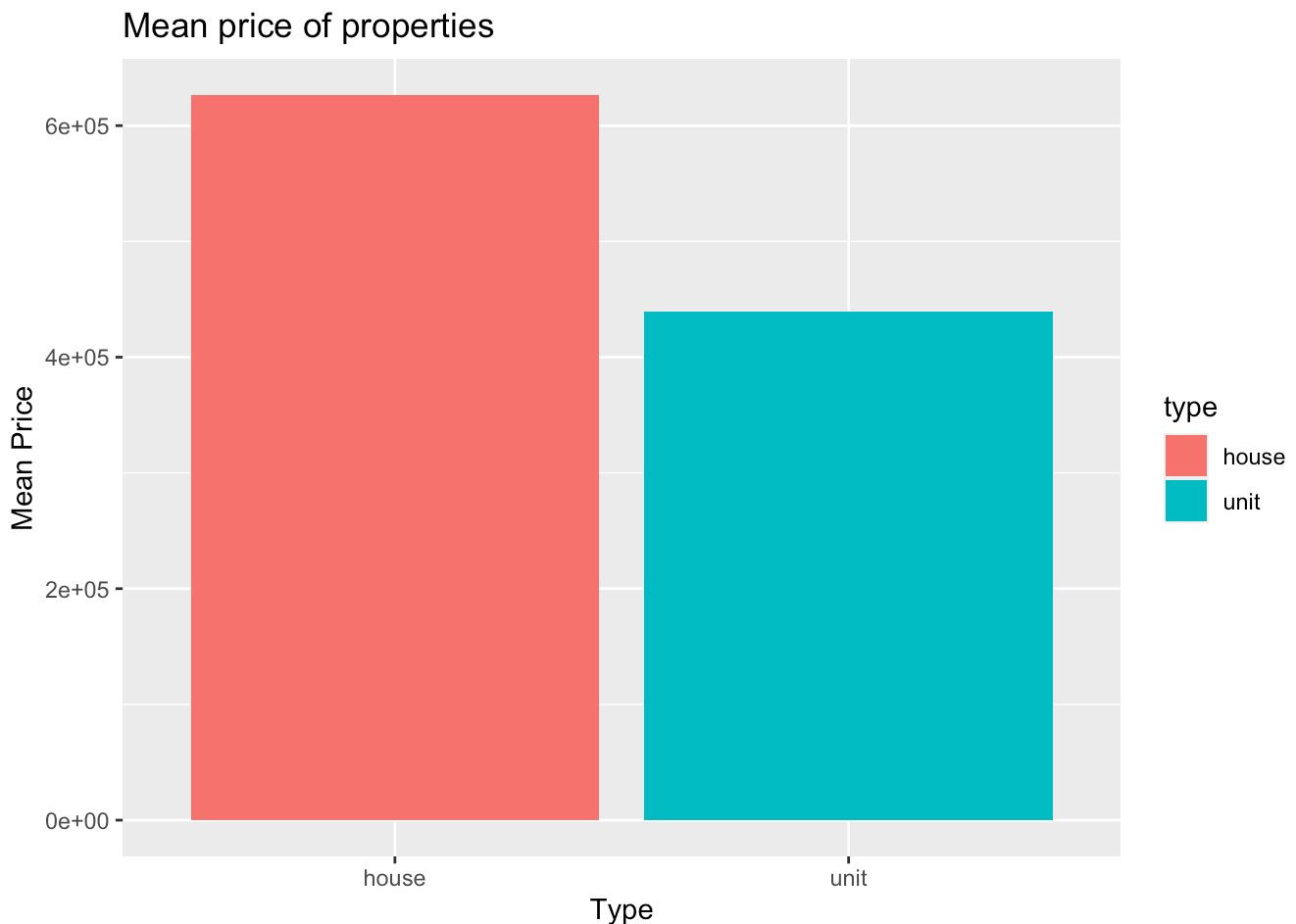
7a. Calculate and show the mean price of the house properties and unit properties.

```
prop %>%  
  group_by(type) %>%  
  summarise(mean_price=mean(price))
```

```
## # A tibble: 2 × 2  
##   type mean_price  
##   <chr>    <dbl>  
## 1 house  626587.  
## 2 unit   439743.
```

7b. Display the two values from 7a as a bar chart. Be sure to include useful information to help understand your chart.

```
prop %>%
  group_by(type) %>%
  summarise(mean_price=mean(price)) %>%
  ggplot(aes(x=type,y=mean_price,fill= type)) +
  geom_bar(stat='identity') +
  labs(x='Type', y='Mean Price', title= 'Mean price of properties')
```



8. Calculate the total income for each of the bedroom types (hint: group by). Sort the output from highest to lowest total income.

```
prop %>%
  group_by.bedrooms) %>%
  summarise(total_income=sum(price)) %>%
  arrange(desc(total_income))
```

```
## # A tibble: 5 × 2
##   bedrooms total_income
##   <fct>      <dbl>
## 1 3          54340744
## 2 2          43868210
## 3 5          41619884.
## 4 4          34159994
## 5 1          15970772
```

9. Which property and bedroom type COMBINATION draws the highest income?

```
prop %>%
  group_by(type, bedrooms) %>%
  summarise(total_income=sum(price)) %>%
  arrange(desc(total_income)) %>%
  head(1)
```

```
## `summarise()` has grouped output by 'type'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 1 × 3
## # Groups:   type [1]
##   type bedrooms total_income
##   <chr> <fct>      <dbl>
## 1 house 5          41619884.
```

10. Run the code below to create a new column “year” which is the year from the sales date. Replace “prop” in the code with whatever you called your data set. You do not need to do anything else in this question after you edit and run the code.

```
prop$year<-format(as.Date(prop$date, format="%d/%m/%Y"), "%Y")
```

```
head(prop)
```

```
## # A tibble: 6 × 5
##   date      price type bedrooms year
##   <chr>      <dbl> <chr> <fct>   <chr>
## 1 30/06/2007 421291 house 3      2007
## 2 30/06/2007 548969 house 4      2007
## 3 30/06/2007 368817 unit  2      2007
## 4 30/06/2008 441854 house 2      2008
## 5 30/06/2008 419628 house 3      2008
## 6 30/06/2008 559580 house 4      2008
```

11. Find the years having a total income of more than \$16,000,000 from selling properties.

```
prop %>%
  group_by(year) %>%
  summarise(total_income=sum(price)) %>%
  filter(total_income >16000000)
```

```
## # A tibble: 3 × 2
##   year total_income
##   <chr>      <dbl>
## 1 2016      16459234
## 2 2017      17077926
## 3 2018      17359589
```

12. How many houses have been sold after 2015 for a price higher than \$1,000,000?

```
prop %>% filter(type=='house' & year>2015 & price>1000000) %>% summarise(n =n())
```

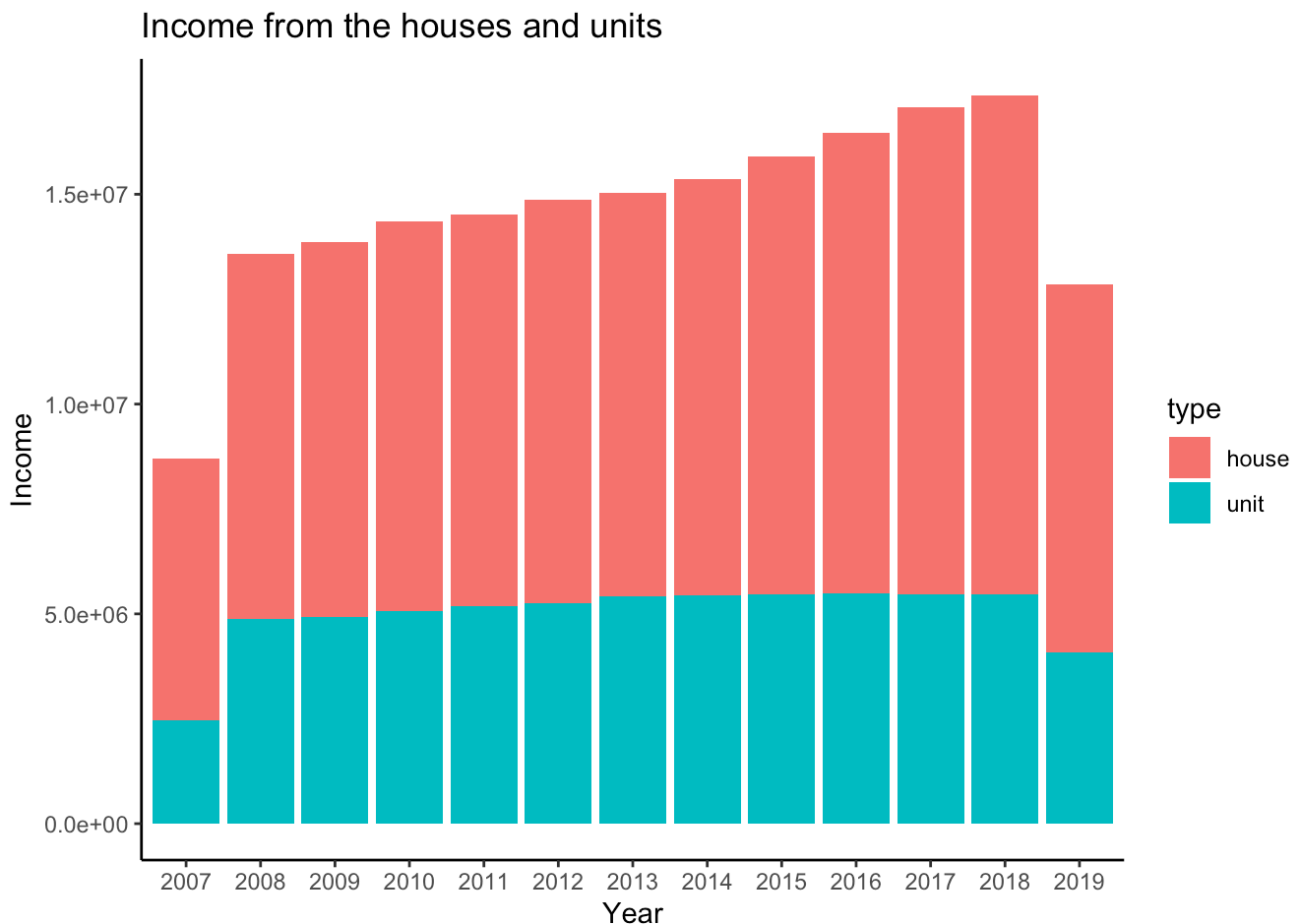
```
## # A tibble: 1 × 1
##       n
##   <int>
## 1     6
```

13. Display the income distribution in each property type over the years using a stacked column chart. Be sure to include useful information

to help your TA understand your chart.

```
prop %>% group_by(year,type) %>% summarise(total_price =sum(price)) %>%
  ggplot(aes(x = year, y = total_price, fill = type)) +
  geom_bar(stat = "identity") + theme_classic() +
  labs( x = "Year", y = "Income", title = "Income from the houses and units")
```

```
## `summarise()` has grouped output by 'year'. You can override using the
## `.groups` argument.
```



14. Find the number of UNIT sales per year for the year range 2010 - 2019. Sort the output from lowest to highest sales.

```
prop%>% filter(type=='unit' & year>=2010 & year<=2019) %>% group_by(year) %>% summarise
(n =n()) %>% arrange(n)
```



```
## # A tibble: 10 × 2
##   year      n
##   <chr> <int>
## 1 2019      9
## 2 2010     12
## 3 2011     12
## 4 2012     12
## 5 2013     12
## 6 2014     12
## 7 2015     12
## 8 2016     12
## 9 2017     12
## 10 2018     12
```